TP4 - Persistent Data

Mobile & Ubiquitous Computing 2026/2027

Ciências, Departamento de Informática

Diogo Branco

Persistent Data

#4 Class

Evaluation

Assignment #3 - Rick and Morty Explorer

0



Storage

Data Storage Types

- App-specific storage
  - Private app internal files
- Shared storage
  - Example photos
- Preferences
  - Store private, primitive data in key-value pairs.
- Databases
  - Local - Room (optional tutorial)
  - Remote - Firebase (todays tutorial)

1



Remote Storage

Firebase - Remote Storage

https://firebase.google.com/codelabs/firebase-android#1

- Step 1, Step 2 (i.e. get sample and understand it)
  - ***Clone or Download Zip***
  - ***Open build-android-start*** *(not the whole project)*
  - ***Skip steps 3, 4 and 5***
- ***Follow Slides***

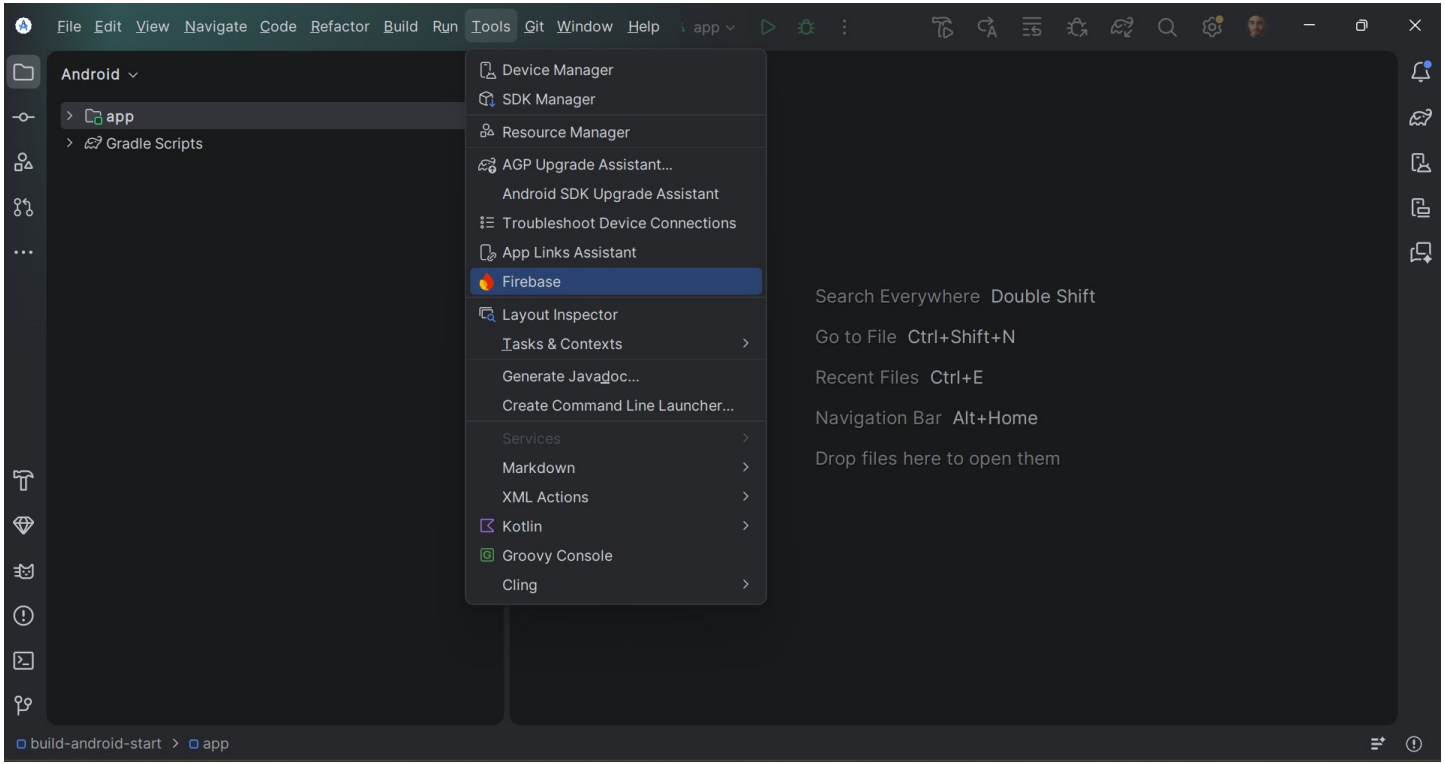
Project does not leverage composables (it is not relevant) for this module use at will

Google Requires Credit Card Info

- **To Use Cloud Storage your project had to be the “pay-as-you go (Blaze) pricing plan** (It will prompt you in the following steps)

- **Then, check if eligible for a $300 credit and free trial Cloud Billing**
  - *Recommend creating a Low-Value (e.g., 1 or 5€) disposable card (in MBWay / Revolut / ...)*
  - *And then Cancel Card* ☺

Follow the steps

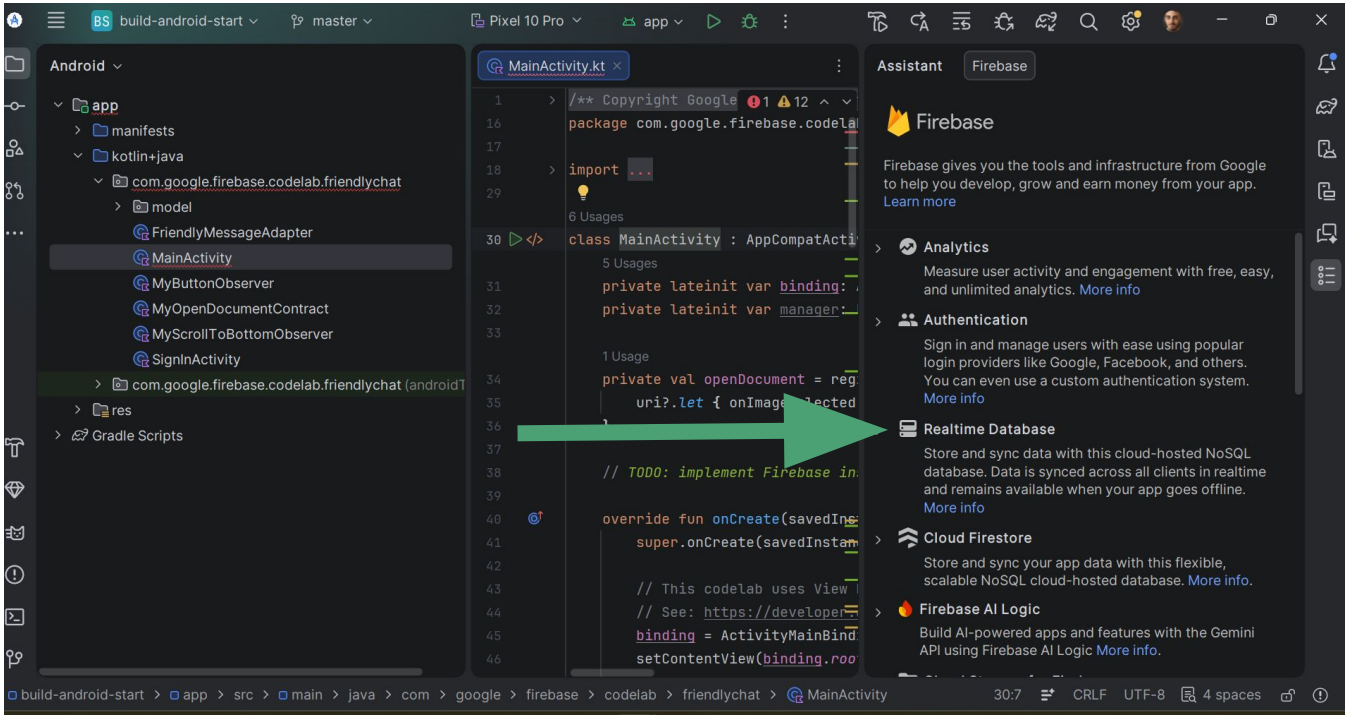


Follow the steps

Do steps 1, 2, 3 of Realtime Database

-Disable analytics

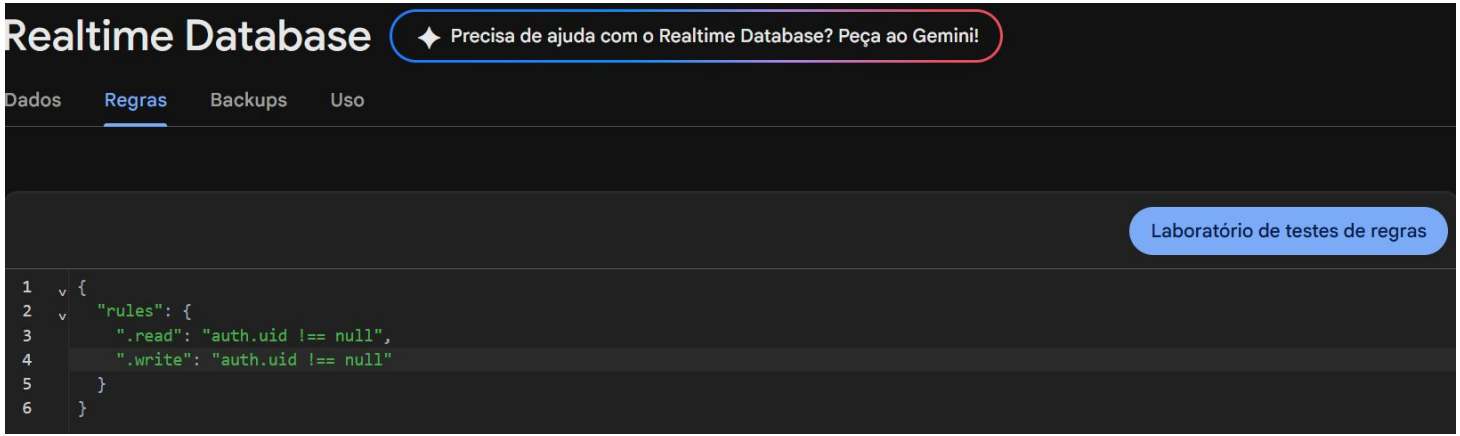
Read the rest*



Don't forget

To create a realtime database instance in firebase (in the browser)

Change the access rules



Firebase - Remote Storage

Do Step 6 (until *test your work)*
https://firebase.google.com/codelabs/firebase-android#1

After Step 6 (*CORRECTIONs)
MainActivity.kt

Rachel Collins *

```
private fun signOut() {
    AuthUI.getInstance().signOut( context: this)
    startActivity(Intent( packageContext: this, SignInActivity::class.java))
```


strings.xml add

```
<string name="default_web_client_id"
translatable="false">webClientId.apps.googleusercontent.com</string>
```

Follow next slide

Firebase - Remote Storage

Remove from import MainActivity

```
import com.google.firebase.codelab.friendlychat.BuildConfig
```

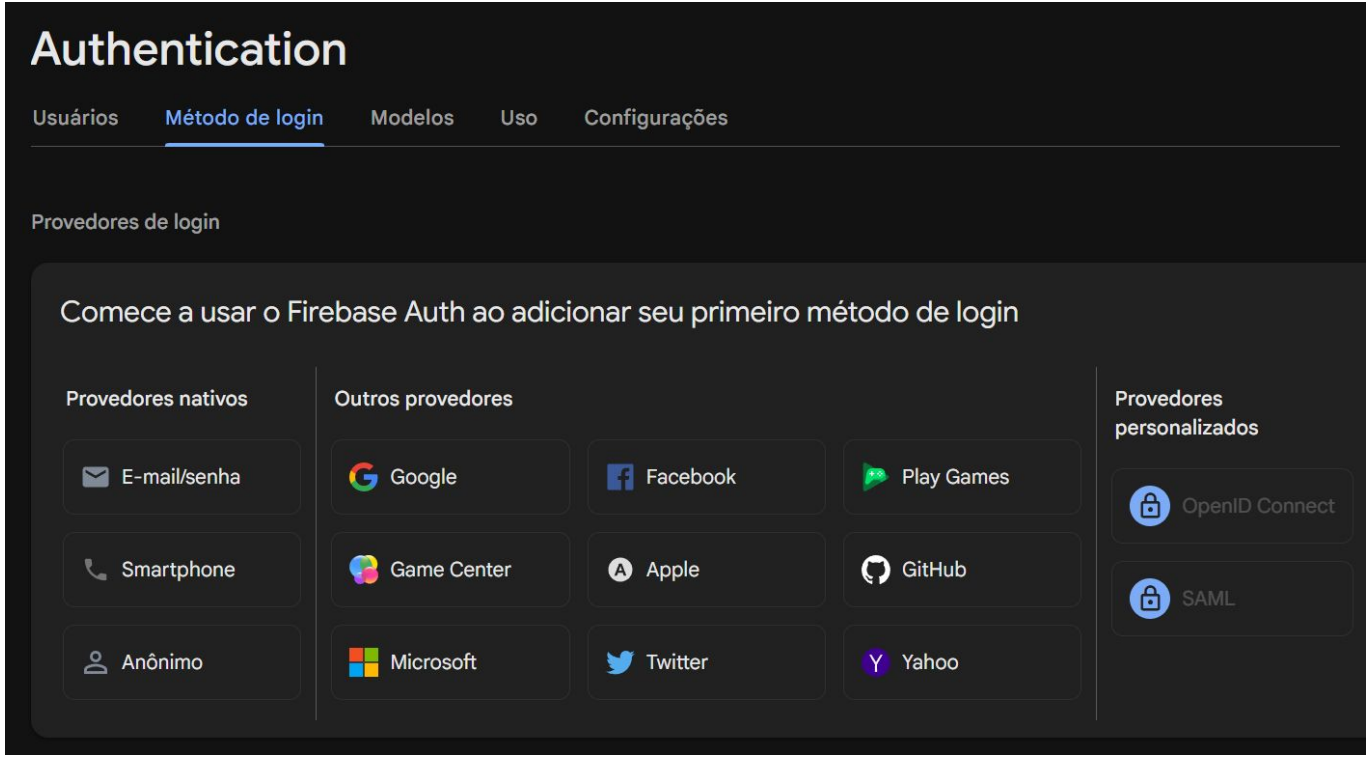
In build.gradle.kts change to

```
implementation("com.google.android.gms:play-services-auth:22.0.0")
```

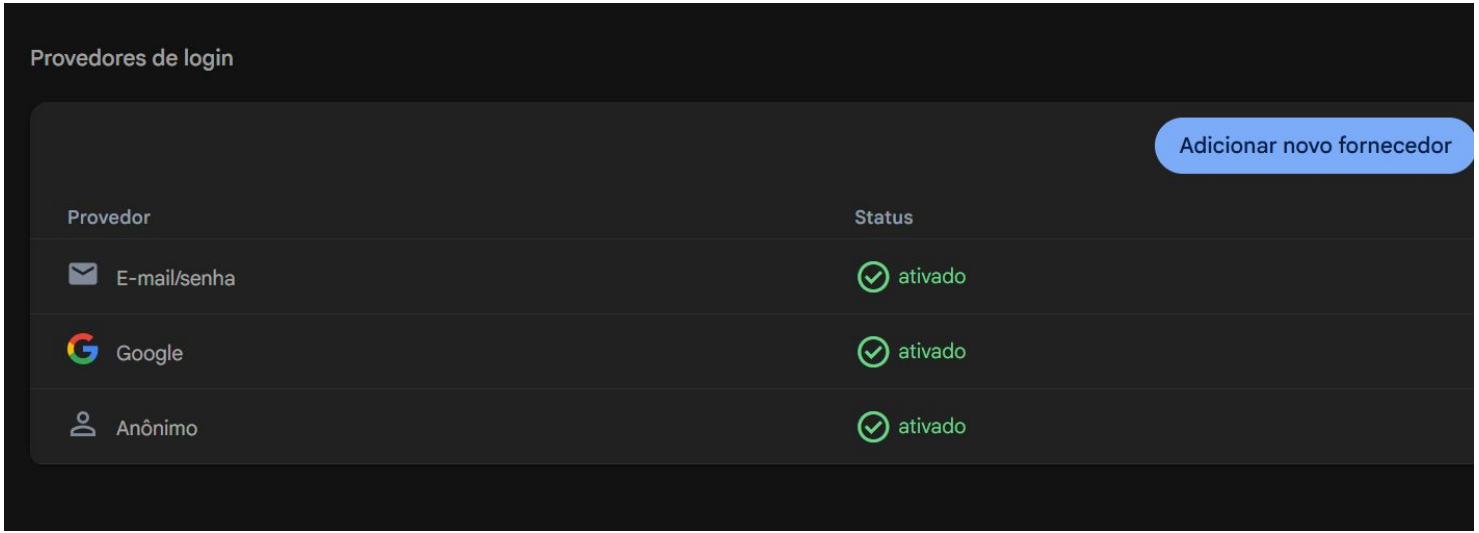
Auth

Go to your firebase console on the browser.

Get Started with authentication



Auth

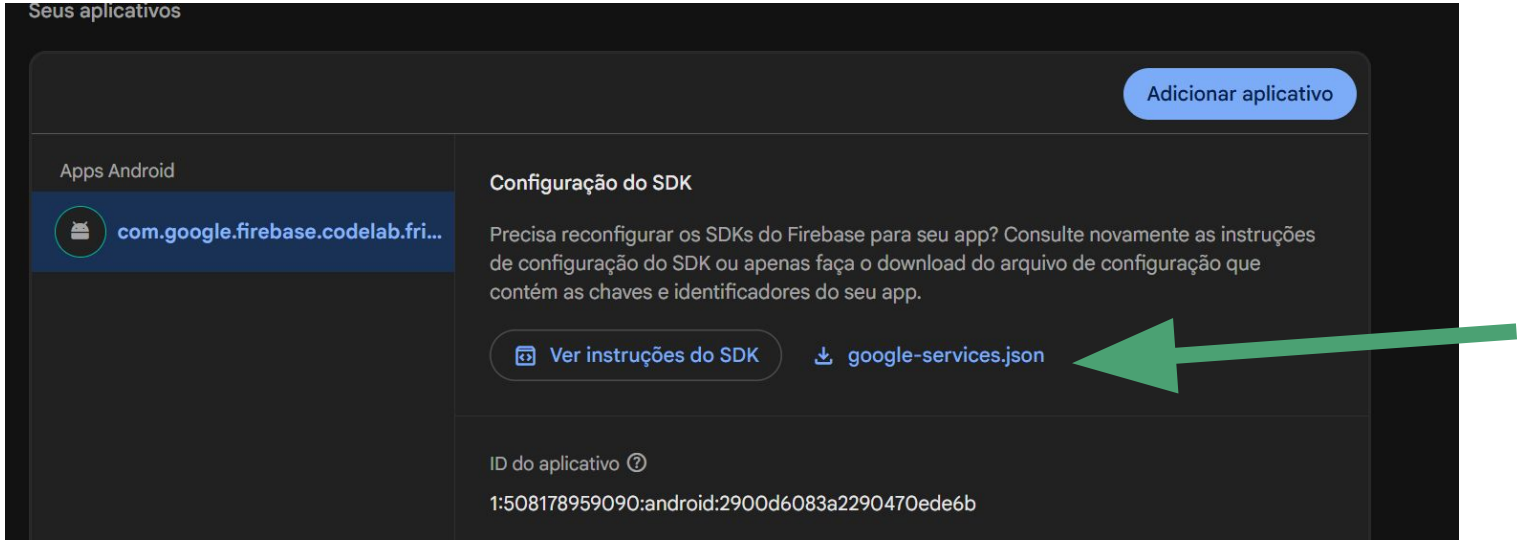

Provedores de login
Adicionar novo fornecedor
Provedor
Status
E-mail/senha
ativado
Google
ativado
Anônimo
ativado

Configs

Go to the settings of the project

Copy the google-services.json file into the app directory of your Android project

(Location may be easier to find in Project Tab rather than Android Tab)

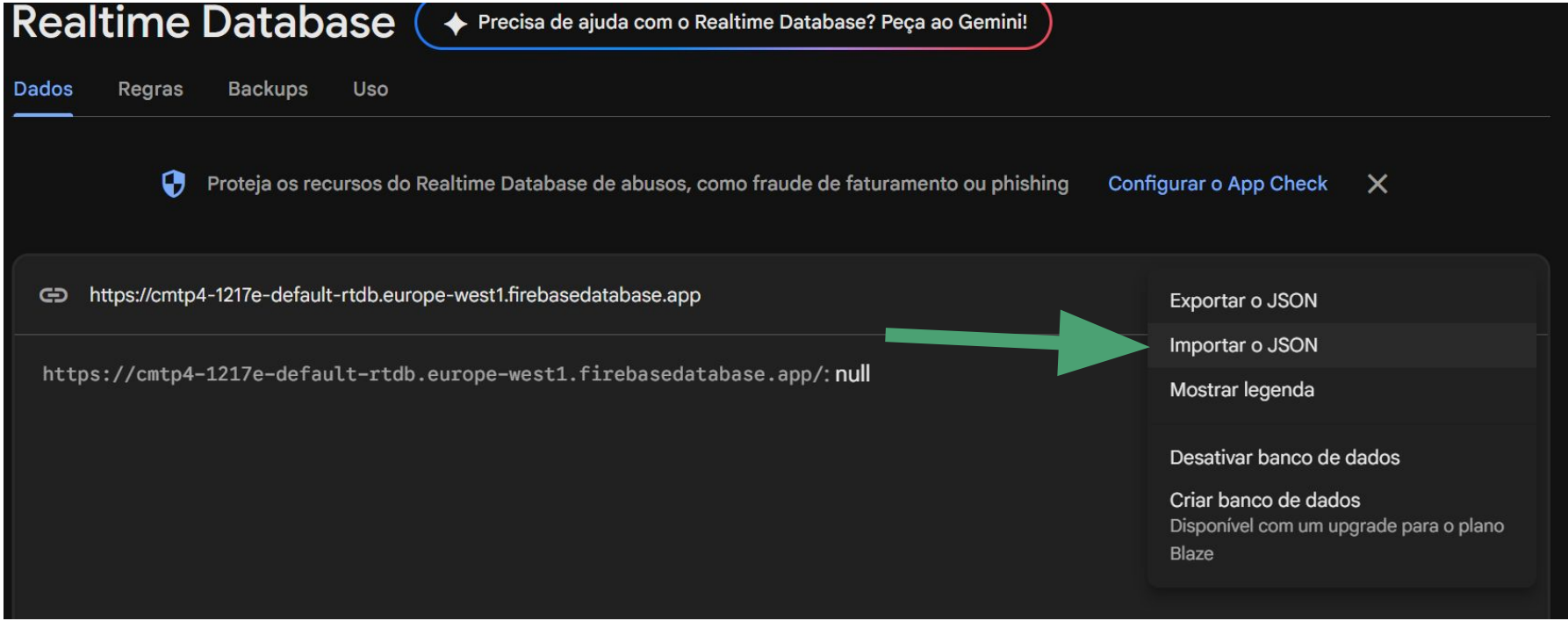


Follow the Steps

- At this point you be able
  - launch the app
  - register a new email
- Next follow slides together with tutorial step 7

Adding messages to the database

Resume step 7 with Import JSON instead of drag and drop



Realtime database rules

At the end of step 7 - you are getting permission denied?
Go to firebase console, realtime database rules and change to (if you havent done before):

```
{
  "rules": {
    "messages": {
      ".read": "auth.uid != null",
      ".write": "auth.uid != null"
    }
  }
}
```

Follow the Steps

- At this point you be able
  - See the messages you added to the database
- Next follow slides together with tutorial step 8

Storage

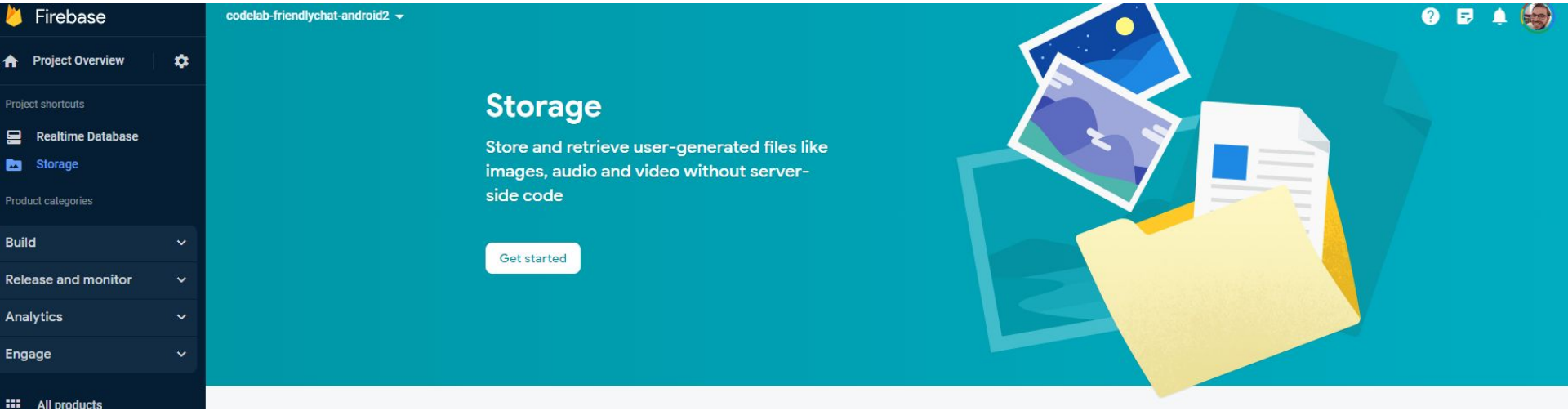
At the end of step 8 you cant click the button?

Add to the end of the onCreate in MainActivity

```
// When the image button is clicked, launch the image picker
        binding.addMessageImageView.setOnClickListener {
            openDocument.launch(arrayOf("image/*"))
        }
```

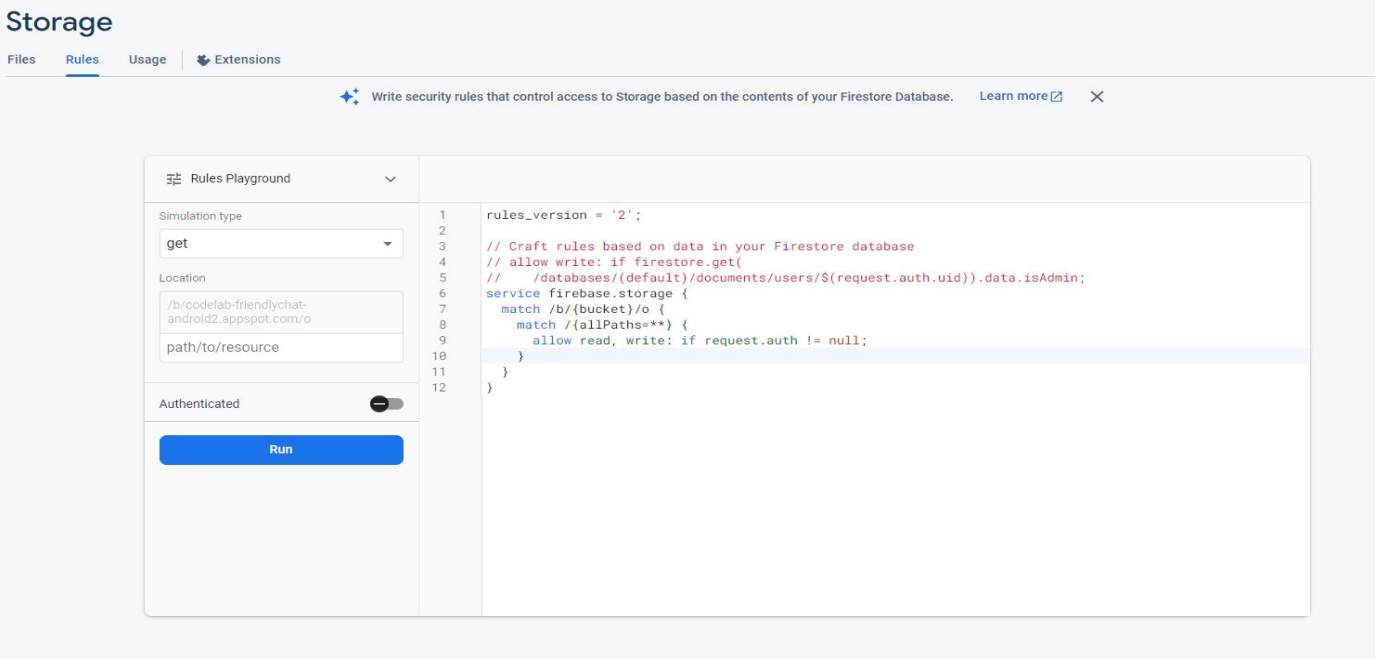
Storage

Go to firebase console and get started with storage:



Storage Rules

Change Storage Rules to allow storage from auth users



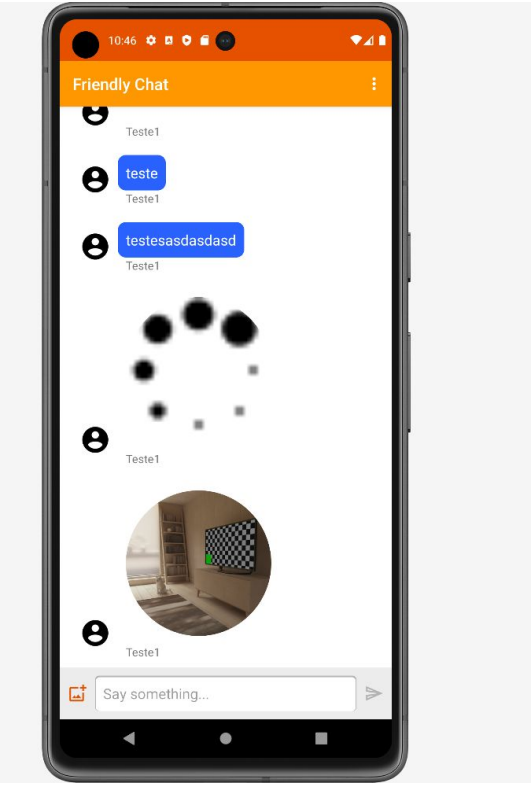
Possible Error

- In MainActivity.kt

```
binding.messageRecyclerView.layoutManager = manager
binding.messageRecyclerView.adapter = adapter
//IF ERROR Inconsistency detected, add this below
binding.messageRecyclerView.itemAnimator = null
```

Working Messaging App

The image should work if taken from the camera*



2



Local Data (Optional)

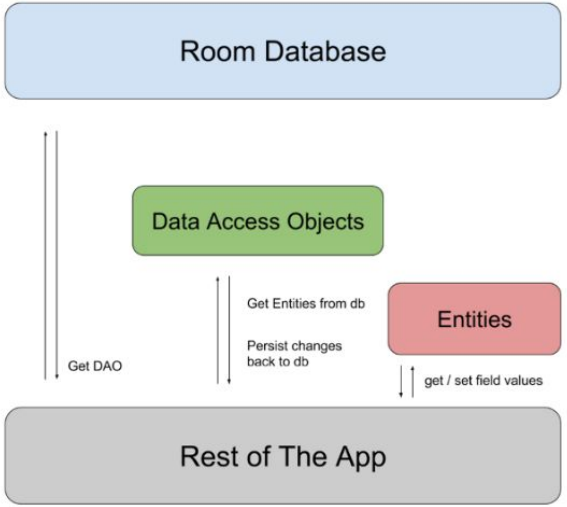
Goals

Interact with the SQLite database using the Room library

Room

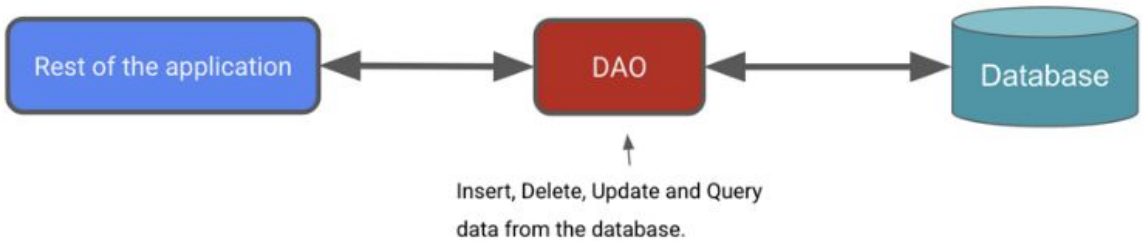
An abstraction layer over SQLite

- Room entities represent tables in your app's database. You use them to update the data stored in rows in tables and to create new rows for insertion
- Room DAOs provide methods that your app uses to retrieve, update, insert, and delete data in the database
- Room Database class is the database class that provides your app with instances of the DAOs associated with that database
- Singleton



Data Access Object (DAO)

- Pattern used to separate the persistence layer
- Hide the complexity of database operations
- Change the data layer independently of the code



Coroutine & Flow

A coroutine is a concurrency design pattern that you can use on Android to simplify code that executes asynchronously.

Flow

- can emit multiple values sequentially, as opposed to suspend functions that return only a single value
- use a flow to receive live updates (e.g. from a database)

Tutorial - Persist Room data **(Optional)**

Download the zip, or make sure you go to the starting branch in git

Follow the tutorial
https://tinyurl.com/CMTPRoom

3



Assignment

Assignment #4 - Remote Storage

Firebase – Distributed storage and communication:

Your application should enable the Rick and Morty API content and its application state to be stored and updated. Your goals are:

1. Add Firebase to your assignment #3 application;
2. Add an option to Save the currently displayed item (Character, Episode, or Location)
   a. It must save the item's name, the image URL and the two chosen relevant schema fields.
   b.
3. Add the option to Load the last saved item back into the application
4. Use the database to store how many times you *Roll*
   a. *Display anywhere how many times you have Rolled*
5. *Use the database to store a history of all items saved over time.*

Tip - No need to use authentication, use the testmode when creating a realtime database.
You can decide to store the image URL, or the image using storage

